

系统开发工具基础 __ 实验报告（第 2 周）

李雨潼

25020007069

一、课上实验题

第 5 题：控制一个可清理的后台任务

题目要求：将给定的 worker.sh 脚本赋予执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。使用 jobs -p 取得 PID，发送 SIGTERM 并等待任务结束。确认 cleanup.log 中出现 CLEAN_EXIT

解题过程：给脚本添加执行权限并前台运行，按 Ctrl+Z 挂起后用 bg 放入后台，用 jobs 确认状态。通过 jobs -p 获取 PID 后 kill 掉，查看 cleanup.log 确认 CLEAN_EXIT

执行结果：

```
dorothea@dorothea-VirtualBox:~/hw1/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
dorothea@dorothea-VirtualBox:~/hw1/q05$ jobs
[1]+ 运行中 ./worker.sh > stdout.log 2> stderr.log &
dorothea@dorothea-VirtualBox:~/hw1/q05$ pid=$(jobs -p)
echo "PID: $pid"
PID: 5335
kill $pid
PID: 5335
dorothea@dorothea-VirtualBox:~/hw1/q05$ jobs
[1]+ 已完成 ./worker.sh > stdout.log 2> stderr.log
dorothea@dorothea-VirtualBox:~/hw1/q05$ cat cleanup.log
CLEAN_EXIT
```

图 1: 第 5 题执行结果

第 6 题：语义重构与本地开发反馈

题目要求：在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成语义重构。分别演示跳转到定义和查找引用。使用”重命名符号”把 total_price 改为 calculate_total，保证定义和所有代码引用同步改变。临时加入未使用的 import os，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

解题过程：创建三个 Python 文件，在 VS Code 中演示跳转到定义和查找引用。用重命名符号将 total_price 改为 calculate_total，验证三个文件同步更新。临时加入未使用的 import os，用 ruff 发现并自动删除。最后运行 pytest 测试通过。

执行结果：

跳转到定义：

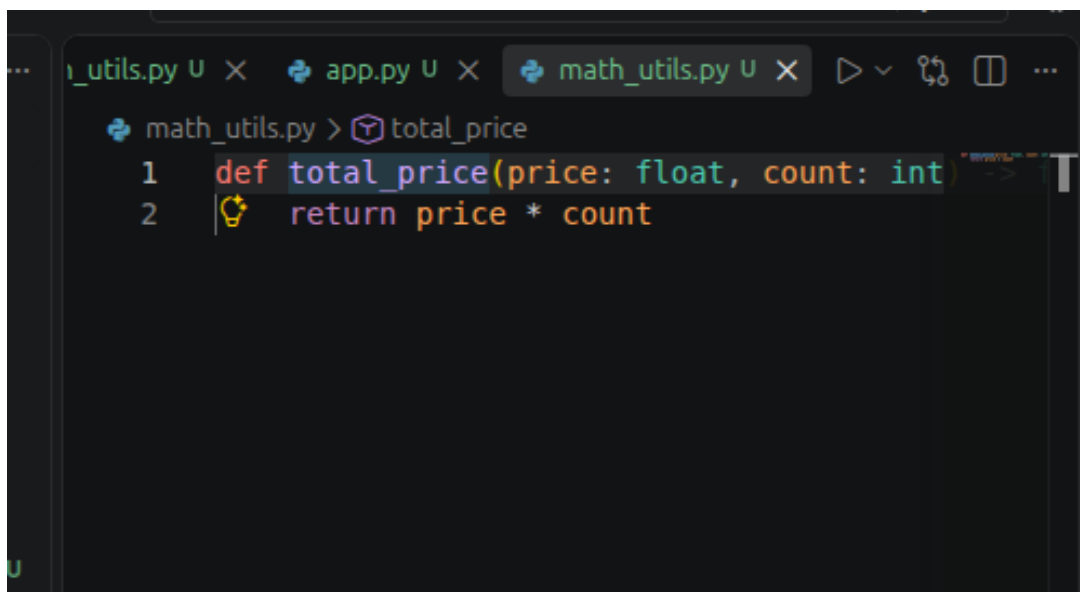


图 2: 跳转到定义

查找引用：

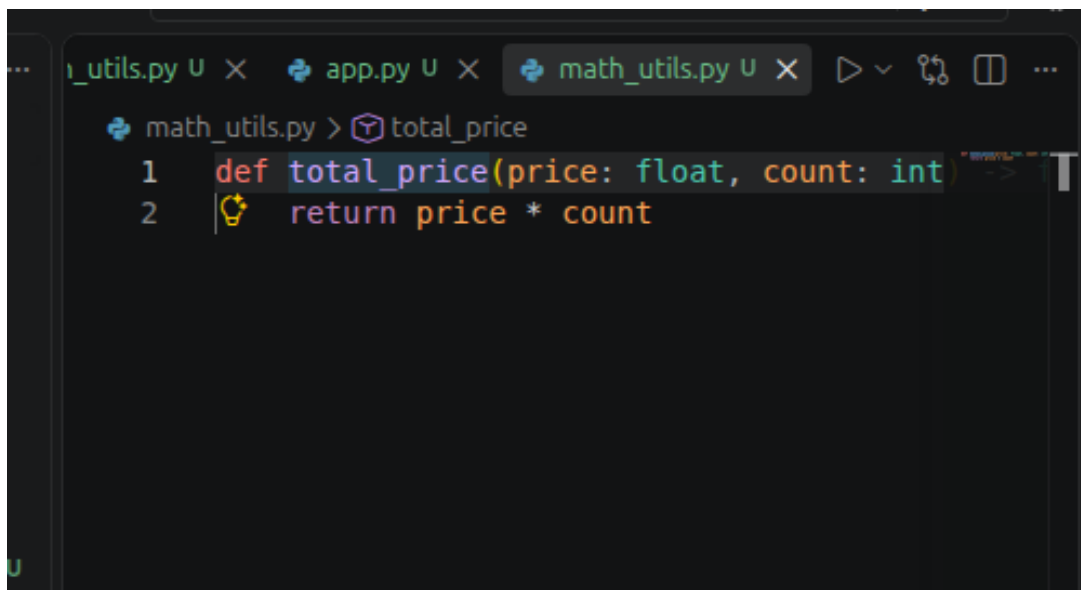


图 3: 查找引用

重命名后三处同步更新:

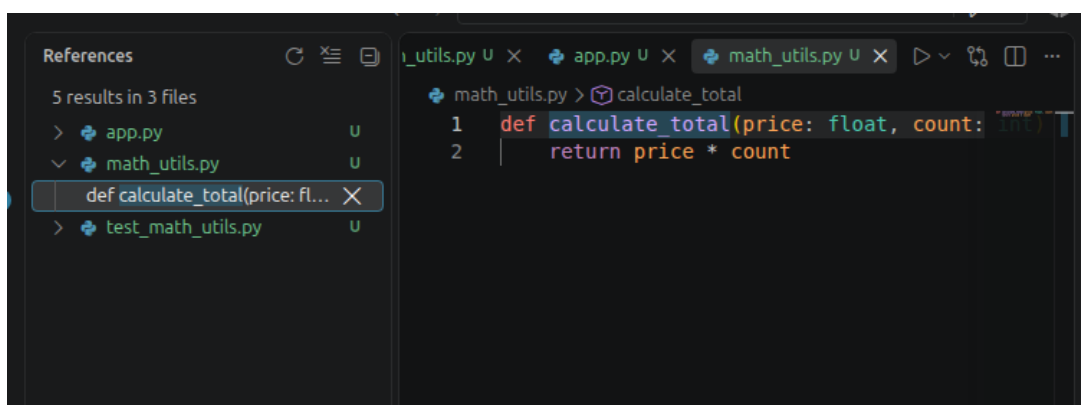


图 4: math_utils.py

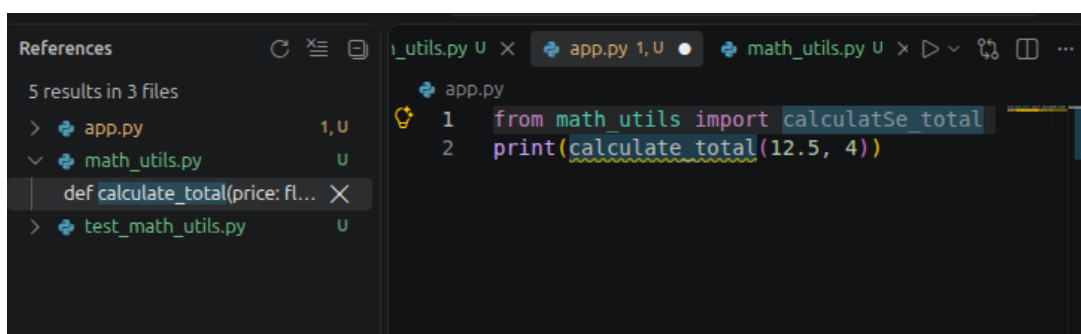


图 5: app.py

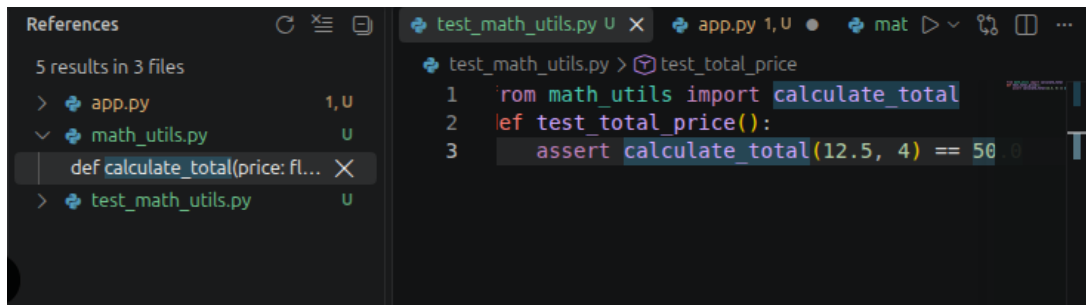


图 6: test_math_utils.py

ruff 检查并修复:

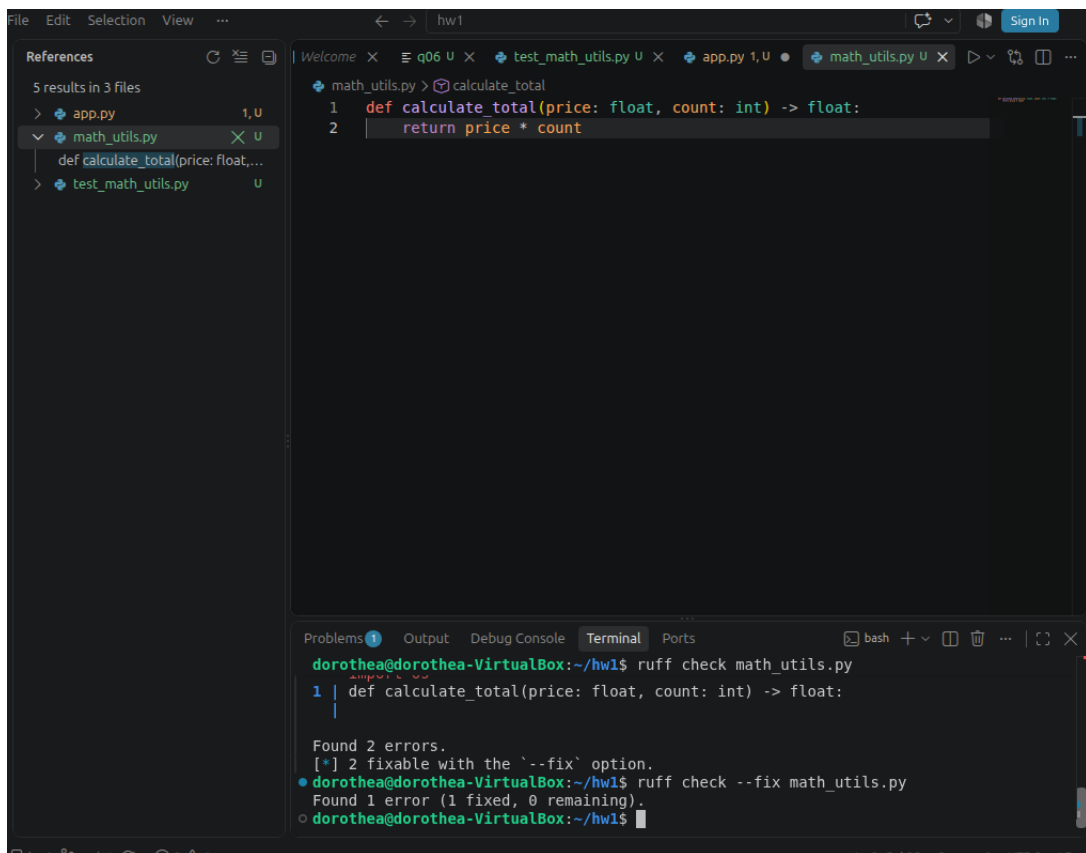
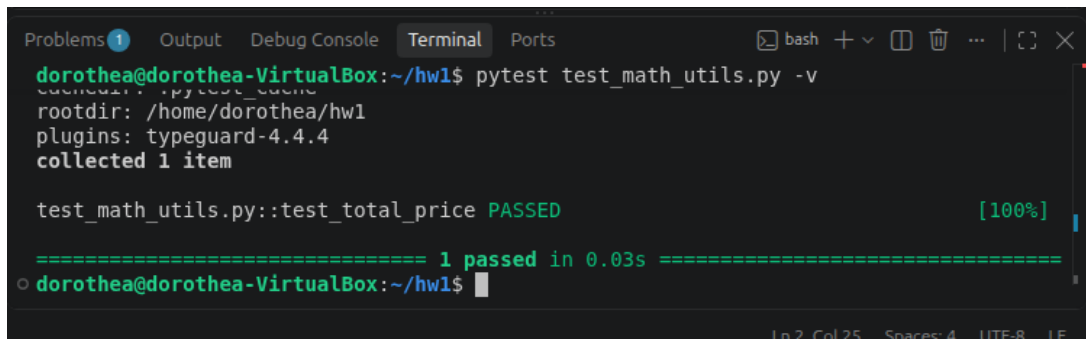


图 7: ruff 检查结果

pytest 测试通过:



The image shows a terminal window with a dark background. At the top, there are tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active, showing a bash shell. The prompt is 'dorothea@dorothea-VirtualBox:~/hw1\$'. The command entered is 'pytest test_math_utils.py -v'. The output shows the pytest configuration: 'rootdir: /home/dorothea/hw1', 'plugins: typeguard-4.4.4', and 'collected 1 item'. The test result is 'test_math_utils.py::test_total_price PASSED' with a green progress bar at [100%]. A summary line reads '===== 1 passed in 0.03s ====='. The prompt returns to 'dorothea@dorothea-VirtualBox:~/hw1\$'.

```
dorothea@dorothea-VirtualBox:~/hw1$ pytest test_math_utils.py -v
rootdir: /home/dorothea/hw1
plugins: typeguard-4.4.4
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.03s =====
dorothea@dorothea-VirtualBox:~/hw1$
```

图 8: pytest 测试通过

第 7 题：用调试器定位并修复排序缺陷

题目要求：将给定代码保存为 `merge_sort.py`，先运行确认结果错误。使用 `pdb` 在 `merge` 中设置断点并观察变量。完成最小修复（不得用 `sorted`），使用 `pytest` 增加两个测试并保证通过。

解题过程：运行发现递归深度超限，将终止条件从 `len(a) < 1` 改为 `<= 1`。再运行发现排序结果仍不对，将 `merge` 函数中的 `right[i]` 改为 `right[j]`。修复后用 `pdb` 设置断点观察变量确认无误。最后写两个 `pytest` 测试用例全部通过。

执行结果：

```
dorothea@dorothea-VirtualBox:~/hw1/q07$ cd ~/hw1/q07
cat > test_merge_sort.py << 'EOF'
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_with_duplicates():
    assert merge_sort([5, 2, 5, 1, 3, 2]) == [1, 2, 2, 3, 5, 5]
EOF
dorothea@dorothea-VirtualBox:~/hw1/q07$ pytest test_merge_sort.py -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/dorothea/hw1/q07
plugins: typeguard-4.4.4
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_with_duplicates PASSED [100%]

===== 2 passed in 0.02s =====
```

图 9: 正确排序结果及 `pytest` 测试

```
dorothea@dorothea-VirtualBox:~/hw1/q07$ cd ~/hw1/q07
python3 -m pdb merge_sort.py
> /home/dorothea/hw1/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b 3
Breakpoint 1 at /home/dorothea/hw1/q07/merge_sort.py:3
(Pdb) c
> /home/dorothea/hw1/q07/merge_sort.py(3)merge()
-> i = j = 0
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p j
*** NameError: name 'j' is not defined
(Pdb) p j
*** NameError: name 'j' is not defined
(Pdb) q
dorothea@dorothea-VirtualBox:~/hw1/q07$
```

图 10: `pdb` 调试过程

第 8 题：先测量再优化慢速词频程序

题目要求：生成 words.txt，使用 time 运行原程序 2 次并记录耗时。使用 cProfile 找出性能瓶颈。在不改变最终 count 的前提下优化程序（不得写死 1000），再运行 2 次，计算加速比。

解题过程：生成 words.txt 后运行原程序，time 测得耗时约 0.119s 和 0.142s。cProfile 分析显示 list 去重是瓶颈，改为 set 去重后耗时降到 0.024s 和 0.032s，加速比约 4.6 倍，count 保持 1000 不变。

执行结果：

```
dorothea@dorothea-VirtualBox: ~/hw1/q08$ cd ~/hw1/q08 && echo "=== 原版运行时间 ===" && time python3 wordfreq.py && echo "" && echo "=== 优化后运行时间 ===" && time python3 wordfreq_optimized.py
=== 原版运行时间 ===
count= 1000

real    0m0.153s
user    0m0.100s
sys     0m0.015s

=== 优化后运行时间 ===
count= 1000

real    0m0.055s
user    0m0.011s
sys     0m0.007s
```

图 11: 原版与优化版运行时间对比

二、课后练习（10 个实例）

实例 1：特殊参数 `-` 的作用

创建和删除以 `-` 开头的文件，验证 `-` 可以避免参数被误解析为选项。

```
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ touch --myfile
error: unexpected argument '--myfile' found

tip: to pass '--myfile' as a value, use '-- --myfile'

Usage: touch [OPTION]... [FILE]...

For more information, try '--help'.
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ touch -- --myfile
ls -la
total 8
-rw-rw-r-- 1 dorothea dorothea  0 Aug 31 16:26 --myfile
drwxrwxr-x 2 dorothea dorothea 4096 Aug 31 16:26 .
drwxrwxr-x 15 dorothea dorothea 4096 Aug 31 16:25 ..
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ rm --myfile
rm: 未识别的选项 "--myfile"
尝试使用"rm ./--myfile"删除文件 '--myfile'。
请尝试执行 "rm --help" 来获取更多信息。
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ rm -- --myfile
ls -la
total 8
drwxrwxr-x 2 dorothea dorothea 4096 Aug 31 16:26 .
drwxrwxr-x 15 dorothea dorothea 4096 Aug 31 16:25 ..
```

图 12: 实例 1 执行结果

实例 2：自定义 `ls` 命令

使用 `-lath -color` 参数列出文件，显示隐藏文件、易读大小、按时间排序、彩色输出。

```
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ ls -lath --color
total 8.0K
drwxrwxr-x 2 dorothea dorothea 4.0K Aug 31 16:26 .
drwxrwxr-x 15 dorothea dorothea 4.0K Aug 31 16:25 ..
```

图 13: 实例 2 执行结果

实例 3：进程替换 + `diff`

用 `diff` 比较 `printenv` 与 `export` 的输出，观察两者的格式差异。


```
;90:*.*tmp=00;90:*.*dpkg-dist=00;90:*.*dpkg-old=00;90:*.*ucf-dist=00;90:*.*ucf-new=00;90:*.*ucf-old=00;90:*.*rpmnew=00;90:*.*rpmorig=00;90:*.*rpmsave=00;90:*"
> declare -x MANAGERPID="2618"
> declare -x MANAGERPIDFID="1360"
> declare -x MEMORY_PRESSURE_WATCH="/sys/fs/cgroup/user.slice/user-1000.slice/user@1000.service/session.slice/dbus.service/memory.pressure"
> declare -x MEMORY_PRESSURE_WRITE="c29tZSAyMDAwMDAgMjAwMDAwMAA="
> declare -x OLDPWD="/home/dorothea/hw1"
> declare -x PATH="/home/dorothea/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin"
> declare -x PTYXIS_PROFILE="b79f483eee839c240df0e46d6a93f150"
> declare -x PTYXIS_VERSION="50.1"
> declare -x PWD="/home/dorothea/hw1/hw2_practice"
> declare -x QT_ACCESSIBILITY="1"
> declare -x QT_IM_MODULE="ibus"
> declare -x QT_IM_MODULE$="wayland;ibus"
> declare -x SHELL="/bin/bash"
> declare -x SHLVL="1"
> declare -x SSH_AUTH_SOCK="/run/user/1000/gcr/ssh"
> declare -x SYSTEMD_EXEC_PID="2660"
> declare -x TERM="xterm-256color"
> declare -x USER="dorothea"
> declare -x USERNAME="dorothea"
> declare -x VTE_VERSION="0400"
> declare -x WAYLAND_DISPLAY="wayland-0"
> declare -x XAUTHORITY="/run/user/1000/.mutter-Xwaylandauth.B2XSU3"
> declare -x XDG_CONFIG_DIRS="/etc/xdg/xdg-ubuntu:/etc/xdg"
> declare -x XDG_CURRENT_DESKTOP="ubuntu:GNOME"
> declare -x XDG_DATA_DIRS="/usr/share/ubuntu:/usr/share/gnome:/usr/local/share:/usr/share:/var/lib/snapd/desktop"
> declare -x XDG_MENU_PREFIX="gnome-"
> declare -x XDG_RUNTIME_DIR="/run/user/1000"
> declare -x XDG_SESSION_CLASS="user"
> declare -x XDG_SESSION_DESKTOP="ubuntu"
> declare -x XDG_SESSION_EXTRA_DEVICE_ACCESS="render:accel"
> declare -x XDG_SESSION_TYPE="wayland"
> declare -x XMODIFIERS="@im=ibus"
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$
```

图 14: 实例 3 执行结果

实例 4: 启用 Vim 模式

在 Shell 中执行 `set -o vi`, 启用 Vim 风格的命令行编辑模式。

```
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ set -o vi
echo "vim mode enabled"
vim mode enabled
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$
```

图 15: 实例 4 执行结果

实例 5: 安装 VS Code 扩展

在 VS Code 中搜索并安装 GitLens 扩展, 增强代码浏览体验。

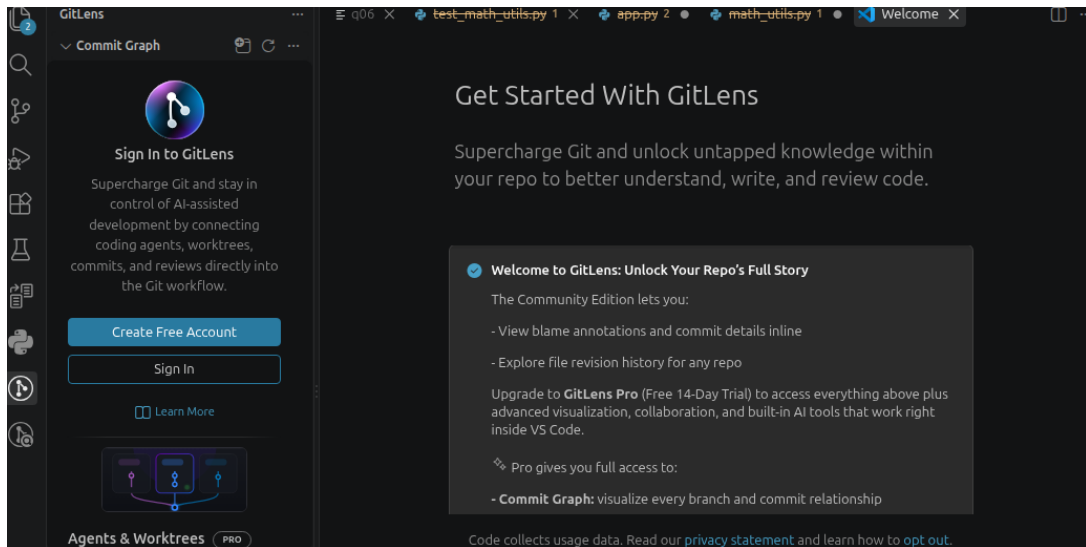


图 16: 实例 5 执行结果

实例 6: 调试归并排序 bug

修复 merge_sort 中 right[i] 应为 right[j] 的索引错误。

```
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ sed -i 's/result.append(right\[i\])/result.append(right\[j\])/' merge_sort.py
python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 17: 实例 6 执行结果

实例 7: AddressSanitizer 检测内存错误

编译时开启 AddressSanitizer, 检测到 heap-use-after-free 错误。

```
dorothea@dorothea-VirtualBox: ~/hw1/hw2_practice
SUMMARY: AddressSanitizer: heap-use-after-free /home/dorothea/hw1/hw2_practice/uaf.c:12 in main
Shadow bytes around the buggy address:
 0x7112535dfd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7112535dfe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7112535dfe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7112535dff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x7112535dff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x7112535e0000: fa fa 00 00 00 fa fa fd fd fd fa fa fa fa
 0x7112535e0080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x7112535e0100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x7112535e0180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x7112535e0200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x7112535e0280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==15377==ABORTING
dorothea@dorothea-VirtualBox: ~/hw1/hw2_practice$
```

图 18: 实例 7 执行结果

实例 8: strace 追踪系统调用

用 `strace -c ls -l` 统计 `ls` 命令调用的系统调用次数和耗时。

% time	seconds	usecs/call	calls	errors	syscall

42.98	0.000496	18	27		mmap
11.87	0.000137	4	34		getdents64
9.36	0.000108	3	33	3	openat
7.54	0.000087	2	34		close
5.37	0.000062	2	29		fstat
3.81	0.000044	2	20		read
3.03	0.000035	5	6		clock_gettime
1.82	0.000021	1	12		statx
1.73	0.000020	5	4	4	connect
1.56	0.000018	2	8		mprotect
1.13	0.000013	2	6		listxattr
1.04	0.000012	6	2		pread64
1.04	0.000012	3	4		socket
0.95	0.000011	5	2		munmap
0.87	0.000010	2	5		ioctl
0.78	0.000009	2	4		newfstatat
0.52	0.000006	1	6		brk
0.52	0.000006	0	9		rt_sigaction
0.43	0.000005	0	7		lseek
0.43	0.000005	1	3		sigaltstack
0.35	0.000004	2	2	2	access
0.35	0.000004	4	1		arch_prctl
0.35	0.000004	4	1		set_tid_address
0.35	0.000004	4	1		set_robust_list
0.35	0.000004	4	1		rseq
0.26	0.000003	3	1		write
0.26	0.000003	3	1		readlink
0.26	0.000003	3	1		sched_getaffinity
0.17	0.000002	2	1		prctl
0.17	0.000002	2	1		gettid
0.17	0.000002	1	2		prlimit64
0.17	0.000002	2	1		getrandom
0.00	0.000000	0	1		poll
0.00	0.000000	0	1		execve
0.00	0.000000	0	3		fcntl
0.00	0.000000	0	2	2	statfs

100.00	0.001154	4	276	11	total

dorothea@dorothea-VirtualBox:~/hw1/hw2_practice\$ S

图 19: 实例 8 执行结果

实例 9: perf 性能分析

用 perf record 采集 slow.c 的运行数据，再用 perf report 查看热点函数。

```

# Samples: 3K of event 'task-clock:ppp'
# Event count (approx.): 960250000
#
# Children      Self  Command  Shared Object      Symbol
# .....
#
# 97.06%    5.10%  slow     slow                [.] main
#      |
#      ---main
#      |
#      --97.03%--slow_computation (inlined)
#          |
#          |--47.80%--__cos_fma
#              |
#              |--26.14%--libc_feholdsetround_sse_ctx (inlined)
#                  |
#                  |--0.78%--stmxcscr_inline_asm (inlined)
#                      |
#                      |--16.01%--do_sincos (inlined)
#                          |
#                          |--8.07%--do_sin (inlined)
#                              |
#                              |--5.49%--do_cos (inlined)
#                                  |
#                                  --3.54%--reduce_sincos (inlined)
#          |
#          --43.92%--__sin_fma
#              |
#              |--26.45%--libc_feholdsetround_sse_ctx (inlined)
#                  |
#                  |--0.65%--asm_sysvec_apic_timer_interrupt
#                      sysvec_apic_timer_interrupt
#              |
#              |--13.23%--do_sincos (inlined)
#                  |
#                  |
# dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$

```

图 20: 实例 9 执行结果

实例 10: 查找并终止占用端口的进程

启动 http.server 占用 4444 端口, 用 ss 查找并 kill 终止。

```

dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$ cd ~/hw1/hw2_practice
python3 -m http.server 4444 &
ss -tlnp | grep 4444
kill $(ss -tlnp | grep 4444 | awk '{print $6}' | cut -d, -f2 | cut -d= -f2)
ss -tlnp | grep 4444
[1] 15542
Serving HTTP on 0.0.0.0 port 4444 (http://0.0.0.0:4444/) ...
LISTEN 0      5            0.0.0.0:4444  0.0.0.0:*
[1]+ 已终止                  python3 -m http.server 4444
dorothea@dorothea-VirtualBox:~/hw1/hw2_practice$

```

图 21: 实例 10 执行结果

三、解题感悟

本次实验涵盖了命令行环境、开发工具、调试和性能分析四个主题。

第 5 题让我熟悉了 Shell 的任务控制和信号处理，通过 `bg`、`jobs`、`kill` 等命令管理后台进程。第 6 题展示了编辑器语言服务器的功能，跳转定义、查找引用和重命名符号提高了代码维护效率。第 7 题的调试练习让我学会了用 `pdb` 逐步跟踪变量状态。第 8 题通过 `cProfile` 和 `time` 直观看到了 `set` 去重相比 `list` 去重的性能提升。

课后 10 个实例补充了 `strace`、`perf`、`AddressSanitizer` 等工具的使用，这些在日常开发和调试中都很实用。整体来说，这周实验让我对各种开发工具有了更具体的认识。